I am completing the FlyRank AI Fluency prompt iteration assignment.

Your job is to run the following six prompts **exactly as written**, independently, and give me the output produced by each one.

Important Instructions

- Run all six versions.
- Treat every version as an independent prompt.
- Do NOT combine information or instructions from later versions into earlier versions.
- Do NOT improve or rewrite any of the prompts.
- Do NOT analyze the prompts.
- Do NOT explain the techniques.
- Do NOT add commentary about the assignment.
- Do NOT invent project facts, metrics, datasets, or results.
- For each version, provide only the natural answer that the prompt itself would produce.
- Keep each output reasonably concise.
- Clearly label each output as Version 0, Version 1, etc. so I can copy them into my assignment.

VERSION 0 — NAIVE PROMPT

Technique: None (baseline)

Prompt:

> Explain how to improve search ranking using machine learning.

Output:
Version 0 — Output

Improving search ranking with machine learning typically involves these steps:

1. **Collect training data**: Gather query-document pairs with relevance labels (from human judgments, click data, or conversions).
2. **Feature engineering**: Extract features like text relevance (TF-IDF, BM25), semantic similarity (embeddings), popularity signals, freshness, and user behavior signals (CTR, dwell time).
3. **Choose a ranking approach**: Use learning-to-rank methods such as pointwise (regression/classification), pairwise (RankNet, LambdaMART), or listwise (ListNet, LambdaRank) models.

4. **Train and validate**: Split data into train/validation/test sets, ensuring no data leakage across queries.
5. **Evaluate with ranking metrics**: Use NDCG, MAP, MRR, or precision@k to assess quality.
6. **Iterate**: Analyze errors, refine features, and experiment with model complexity (e.g., gradient-boosted trees like LightGBM/XGBoost, or neural rankers) based on gains.
7. **Deploy and monitor**: Roll out gradually (A/B testing), and monitor for regressions or drift in production.

The key is to start with strong baselines and simple models before increasing complexity, ensuring each improvement is measurable.

VERSION 1 — ROLE ASSIGNMENT

Technique: Role assignment

Prompt:

> Act as an ML engineer mentoring an intern. Explain how to approach improving search ranking using machine learning.

Output:
Version 1 — Output

As your mentor, here's how I'd guide you through improving search ranking:

Step 1: Understand the current system. Before touching any model, know what ranking signals and features are already in use, and what evaluation metric matters (NDCG, MRR, etc.).

Step 2: Establish a strong baseline. Make sure you have a simple, well-understood baseline (e.g., BM25 or a basic linear model) so you can measure real improvement.

Step 3: Diagnose weaknesses. Look at where the current system fails — specific query types, missing features, noisy labels, or data imbalance. Don't assume the model architecture is the bottleneck; often it's the data or features.

Step 4: Improve incrementally. Start with feature engineering (better relevance signals, user behavior features) before jumping to complex models like deep neural rankers.

Step 5: Validate rigorously. Use held-out data and proper ranking metrics. Watch for data leakage, especially with time-based or user-based splits.

Step 6: Only add complexity when justified. Move to gradient-boosted trees (LambdaMART) or neural approaches only if simpler methods plateau and you have enough quality data to support them.

The mentoring takeaway: as an intern, resist the urge to reach for the most sophisticated model first — most ranking gains come from better data and features, not model complexity.

VERSION 2 — CONTEXT AND MOTIVATION

Technique: Context and motivation

Prompt:

> Act as an ML engineer mentoring an intern. I am working on a FlyRank internship task focused on Search Ranking Improvement. The existing ranking pipeline has a baseline performance of about 0.24 compared with a reference model around 0.74. I want to understand the problem and decide what should be investigated before choosing a complex model. Explain how I should approach it.

Output:
Version 2 — Output

Good question — let's think through this systematically before jumping to solutions.

Understand the gap first. A baseline of 0.24 versus a reference model at 0.74 is a large gap. Before assuming the reference model is simply "better," you need to understand *why*. Possible causes include: better features, better training data, better labels, or genuinely superior model architecture.

Check the fundamentals:

- What metric produced these numbers? Confirm it's the same metric, computed the same way, on the same test set — mismatches here can create misleading gaps.
- What features does the baseline use versus the reference model? Often a large gap comes from missing or weak features, not the algorithm itself.
- What does the baseline model actually look like? If it's very simple (e.g., a naive heuristic), a large gap might just reflect that difference, not a model complexity issue.

Investigate before modeling:

- Data quality: Are relevance labels reliable and consistent?
- Feature coverage: Does the baseline miss important ranking signals present in the reference model?
- Evaluation setup: Are both models being tested identically (same queries, splits, and metric implementation)?

Then decide on next steps. Only after this diagnosis should you decide whether to improve features, fix data issues, or introduce a more complex model. Jumping straight to a complex model risks masking a data/feature problem rather than solving it.

As your mentor, my advice: treat this gap as a diagnostic puzzle first, and a modeling challenge second.

VERSION 3 — FEW-SHOT EXAMPLES

Technique: Few-shot examples

Prompt:

> Act as an ML engineer mentoring an intern. I am working on a FlyRank internship task focused on Search Ranking Improvement. The existing ranking pipeline has a baseline performance of about 0.24 compared with a reference model around 0.74. I want to understand the problem and decide what should be investigated before choosing a complex model.
>
> Examples of the type of reasoning I want:
>
> Example 1: Problem: Search ranking performance is below the target. Good approach: First identify where the ranking pipeline loses performance before changing the model.
>
> Example 2: Problem: A model has poor evaluation results. Good approach: Check the data, features, baseline and evaluation process before assuming the model itself is the problem.
>
> Now explain how I should approach the FlyRank ranking problem.

Output:
Version 3 — Output

Following the same reasoning style as the examples:

Problem: The FlyRank ranking pipeline has a baseline performance of 0.24, far below the reference model's 0.74.

Good approach: Before changing the model, identify where the pipeline is losing performance. This means checking the data, features, baseline setup, and evaluation process — not assuming the model architecture itself is the cause.

Applying this to your case:

1. **Verify the evaluation is apples-to-apples.** Confirm both models are evaluated on the same metric, same test set, and same computation method.
2. **Examine the baseline's features and data.** A 0.24 vs 0.74 gap often reflects missing signals (e.g., no semantic similarity, no user behavior features) rather than a fundamentally weak model.
3. **Check data/label quality.** Noisy or sparse relevance labels can cap performance regardless of model choice.
4. **Look at the reference model's design.** Understand what it does differently — more features, more data, or a different ranking approach — to know what's actually driving its advantage.
5. **Only after this diagnosis**, decide whether the fix is better features, better data, or a more complex model.

This mirrors the example reasoning: diagnose the source of the gap first, then decide on the right level of model complexity.

VERSION 4 — OUTPUT STRUCTURE

Technique: Output structure

Prompt:

> Act as an ML engineer mentoring an intern. I am working on a FlyRank internship task focused on Search Ranking Improvement. The existing ranking pipeline has a baseline performance of about 0.24 compared with a reference model around 0.74. I want to understand the problem and decide what should be investigated before choosing a complex model.
>
> Examples of the reasoning style I want:
>
> - First understand the source of a performance gap.
> - Establish a baseline before making changes.
> - Investigate data and features before assuming the model is the problem.
>
> Structure your answer using exactly these four sections:
>
> 1. Problem
> 2. Evidence
> 3. Investigation

4. Recommended Next Step

Output:

Version 4 — Output

1. Problem

The FlyRank search-ranking pipeline currently performs well below a reference model (0.24 vs. 0.74), and the goal is to understand the cause of this gap before selecting a more complex model.

2. Evidence

- Baseline performance: ~0.24
- Reference model performance: ~0.74
- This is a substantial gap, suggesting either a significant capability difference or a data/feature/evaluation issue.

3. Investigation

- Confirm the evaluation metric and setup are identical for both models (same test set, same metric computation).
- Review the baseline's features: is it missing key ranking signals (semantic relevance, user behavior, popularity) that the reference model uses?
- Check data and label quality: are relevance judgments reliable and consistent?
- Examine whether the gap is uniform across query types or concentrated in specific segments (e.g., rare queries).
- Investigate the reference model's design to understand what's driving its performance — richer features, more data, or a different algorithm.

4. Recommended Next Step

Conduct a diagnostic analysis of data quality, feature coverage, and evaluation consistency before choosing or building a more complex model. This ensures any modeling effort addresses the actual bottleneck rather than masking a data or feature limitation.

VERSION 5 — STEP DECOMPOSITION

Technique: Step decomposition

Prompt:

> Act as an ML engineer mentoring an intern. I am working on a FlyRank internship task focused on Search Ranking Improvement. The existing ranking pipeline has a baseline performance of about 0.24 compared with a reference model around 0.74.

I want to understand the problem and decide what should be investigated before choosing a complex model.

Examples of the reasoning style I want: - Diagnose the source of a performance gap before changing the model. - Establish a baseline before making improvements. - Investigate data and features before assuming the model is the problem.

Structure the answer using: 1. Problem 2. Evidence 3. Investigation 4. Recommended Next Step

Work through the task in this order: 1. Define the ranking problem. 2. Identify what the baseline tells us. 3. Identify what information is still missing. 4. List the most important things to investigate. 5. Recommend one immediate next step.

Do not invent metrics, dataset details or model results. Clearly separate known information from assumptions.

Output:

Version 5 — Output

1. Problem

The FlyRank ranking pipeline's baseline model underperforms significantly compared to a reference model, and the task is to determine what's causing this gap before considering more complex modeling approaches.

2. Evidence

Known (from context): Baseline performance is around 0.24; reference model performance is around 0.74.

Unknown: The specific metric used, dataset characteristics, feature sets used by each model, and evaluation methodology are not specified.

3. Investigation

Things to investigate, in order:

- **Evaluation consistency:** Confirm both models were evaluated on the same metric, same test set, and same scoring method — mismatches here can produce misleading gaps.
- **Feature coverage:** Determine what features the baseline uses versus the reference model; missing signals are a common source of large gaps.
- **Data and label quality:** Assess whether relevance labels are reliable, consistent, and sufficiently representative.
- **Error distribution:** Check whether poor performance is spread evenly or concentrated in specific query types.
- **Reference model design:** Understand what makes the reference model stronger — additional data, richer features, or different architecture — since this is currently unknown.

4. Recommended Next Step

Assumption/recommendation (not a stated fact): Before selecting a more complex model,

perform a diagnostic review of evaluation setup, feature coverage, and data quality to isolate the actual cause of the performance gap. This step should determine whether the next move is data/feature improvement or a genuine model upgrade.

FINAL RESPONSE FORMAT

Version 0 — Output

Improving search ranking with machine learning typically involves these steps:

1. **Collect training data**: Gather query-document pairs with relevance labels (from human judgments, click data, or conversions).
2. **Feature engineering**: Extract features like text relevance (TF-IDF, BM25), semantic similarity (embeddings), popularity signals, freshness, and user behavior signals (CTR, dwell time).
3. **Choose a ranking approach**: Use learning-to-rank methods such as pointwise (regression/classification), pairwise (RankNet, LambdaMART), or listwise (ListNet, LambdaRank) models.
4. **Train and validate**: Split data into train/validation/test sets, ensuring no data leakage across queries.
5. **Evaluate with ranking metrics**: Use NDCG, MAP, MRR, or precision@k to assess quality.
6. **Iterate**: Analyze errors, refine features, and experiment with model complexity (e.g., gradient-boosted trees like LightGBM/XGBoost, or neural rankers) based on gains.
7. **Deploy and monitor**: Roll out gradually (A/B testing), and monitor for regressions or drift in production.

The key is to start with strong baselines and simple models before increasing complexity, ensuring each improvement is measurable.

Version 1 — Output

As your mentor, here's how I'd guide you through improving search ranking:

Step 1: Understand the current system. Before touching any model, know what ranking signals and features are already in use, and what evaluation metric matters (NDCG, MRR, etc.).

Step 2: Establish a strong baseline. Make sure you have a simple, well-understood baseline (e.g., BM25 or a basic linear model) so you can measure real improvement.

Step 3: Diagnose weaknesses. Look at where the current system fails — specific query types, missing features, noisy labels, or data imbalance. Don't assume the model architecture is the bottleneck; often it's the data or features.

Step 4: Improve incrementally. Start with feature engineering (better relevance signals, user behavior features) before jumping to complex models like deep neural rankers.

Step 5: Validate rigorously. Use held-out data and proper ranking metrics. Watch for data leakage, especially with time-based or user-based splits.

Step 6: Only add complexity when justified. Move to gradient-boosted trees (LambdaMART) or neural approaches only if simpler methods plateau and you have enough quality data to support them.

The mentoring takeaway: as an intern, resist the urge to reach for the most sophisticated model first — most ranking gains come from better data and features, not model complexity.

Version 2 — Output

Good question — let's think through this systematically before jumping to solutions.

Understand the gap first. A baseline of 0.24 versus a reference model at 0.74 is a large gap. Before assuming the reference model is simply "better," you need to understand *why*. Possible causes include: better features, better training data, better labels, or genuinely superior model architecture.

Check the fundamentals:

- What metric produced these numbers? Confirm it's the same metric, computed the same way, on the same test set — mismatches here can create misleading gaps.
- What features does the baseline use versus the reference model? Often a large gap comes from missing or weak features, not the algorithm itself.
- What does the baseline model actually look like? If it's very simple (e.g., a naive heuristic), a large gap might just reflect that difference, not a model complexity issue.

Investigate before modeling:

- Data quality: Are relevance labels reliable and consistent?
- Feature coverage: Does the baseline miss important ranking signals present in the reference model?
- Evaluation setup: Are both models being tested identically (same queries, splits, and metric implementation)?

Then decide on next steps. Only after this diagnosis should you decide whether to improve features, fix data issues, or introduce a more complex model. Jumping straight to a complex model risks masking a data/feature problem rather than solving it.

As your mentor, my advice: treat this gap as a diagnostic puzzle first, and a modeling challenge second.

Version 3 — Output

Following the same reasoning style as the examples:

Problem: The FlyRank ranking pipeline has a baseline performance of 0.24, far below the reference model's 0.74.

Good approach: Before changing the model, identify where the pipeline is losing performance. This means checking the data, features, baseline setup, and evaluation process — not assuming the model architecture itself is the cause.

Applying this to your case:

1. **Verify the evaluation is apples-to-apples.** Confirm both models are evaluated on the same metric, same test set, and same computation method.
2. **Examine the baseline's features and data.** A 0.24 vs 0.74 gap often reflects missing signals (e.g., no semantic similarity, no user behavior features) rather than a fundamentally weak model.
3. **Check data/label quality.** Noisy or sparse relevance labels can cap performance regardless of model choice.
4. **Look at the reference model's design.** Understand what it does differently — more features, more data, or a different ranking approach — to know what's actually driving its advantage.
5. **Only after this diagnosis**, decide whether the fix is better features, better data, or a more complex model.

This mirrors the example reasoning: diagnose the source of the gap first, then decide on the right level of model complexity.

Version 4 — Output

1. Problem The FlyRank search-ranking pipeline currently performs well below a reference model (0.24 vs. 0.74), and the goal is to understand the cause of this gap before selecting a more complex model.

2. Evidence

- Baseline performance: ~0.24
- Reference model performance: ~0.74

- This is a substantial gap, suggesting either a significant capability difference or a data/feature/evaluation issue.

3. Investigation

- Confirm the evaluation metric and setup are identical for both models (same test set, same metric computation).
- Review the baseline's features: is it missing key ranking signals (semantic relevance, user behavior, popularity) that the reference model uses?
- Check data and label quality: are relevance judgments reliable and consistent?
- Examine whether the gap is uniform across query types or concentrated in specific segments (e.g., rare queries).
- Investigate the reference model's design to understand what's driving its performance — richer features, more data, or a different algorithm.

4. Recommended Next Step Conduct a diagnostic analysis of data quality, feature coverage, and evaluation consistency before choosing or building a more complex model. This ensures any modeling effort addresses the actual bottleneck rather than masking a data or feature limitation.

Version 5 — Output

1. Problem The FlyRank ranking pipeline's baseline model underperforms significantly compared to a reference model, and the task is to determine what's causing this gap before considering more complex modeling approaches.

2. Evidence Known (from context): Baseline performance is around 0.24; reference model performance is around 0.74. Unknown: The specific metric used, dataset characteristics, feature sets used by each model, and evaluation methodology are not specified.

3. Investigation Things to investigate, in order:

- **Evaluation consistency:** Confirm both models were evaluated on the same metric, same test set, and same scoring method — mismatches here can produce misleading gaps.
- **Feature coverage:** Determine what features the baseline uses versus the reference model; missing signals are a common source of large gaps.
- **Data and label quality:** Assess whether relevance labels are reliable, consistent, and sufficiently representative.
- **Error distribution:** Check whether poor performance is spread evenly or concentrated in specific query types.
- **Reference model design:** Understand what makes the reference model stronger — additional data, richer features, or different architecture — since this is currently unknown.

4. Recommended Next Step Assumption/recommendation (not a stated fact): Before selecting a more complex model, perform a diagnostic review of evaluation setup, feature coverage, and

data quality to isolate the actual cause of the performance gap. This step should determine whether the next move is data/feature improvement or a genuine model upgrade.